GIT · CONFLICTOS DE CÓDIGO

Conflictos de código

Detectar · entender · resolver · verificar

Guía visual y práctica para salir de un merge, rebase, cherry-pick o stash sin perder cambios.

10 EJERCICIOS

57 PASS · 0 FAIL

GIT 2.56

La evidencia real de la práctica

```
$ git merge feature
Auto-merging app.txt
CONFLICT (content):
Merge conflict in app.txt

$ git status --short
UU app.txt

$ git add app.txt
$ git commit -m "merge: ..."

[PASS] árbol limpio
RESULTADO: 5 PASS, 0 FAIL
```

EN 20 DIAPOSITIVAS

Diagnóstico · resolución · verificación

10 ejercicios reales, no solo teoría.

El conflicto no se adivina: se diagnostica.

Ivan · Octubre 2026

github.com/75749748-blip/conflictos-git-ejercicios

El conflicto no es un error

Es una decisión que Git no puede tomar por ti.

Dos historias

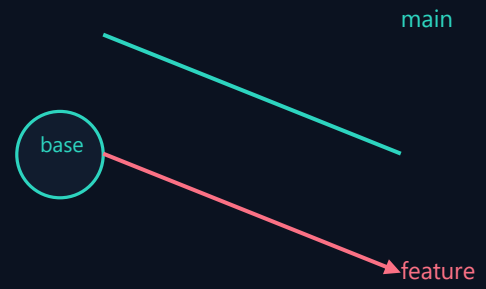
Dos ramas cambiaron el mismo archivo desde una base común.

Misma región

Los cambios se solapan y Git no sabe cuál conservar o cómo combinarlos.

Decisión humana

Una persona lee el contexto, elige el resultado y lo valida.



merge

Git se detiene y marca el archivo.

Sin conflicto: Git combina automáticamente.
Con conflicto: Git pide una decisión.

Leer los marcadores

El archivo te muestra las tres versiones. No borres a ciegas.

diff3 / zdiff3

```
linea 1
<<<<<<< HEAD
linea 2 (main)
||||||| base
linea 2
=======
linea 2 (feature)
>>>>>>> feature
linea 3
```

<<<<<<< HEAD

ours: tu rama actual

||||||| base

base común (diff3)

=======

separador

>>>>>>> feature

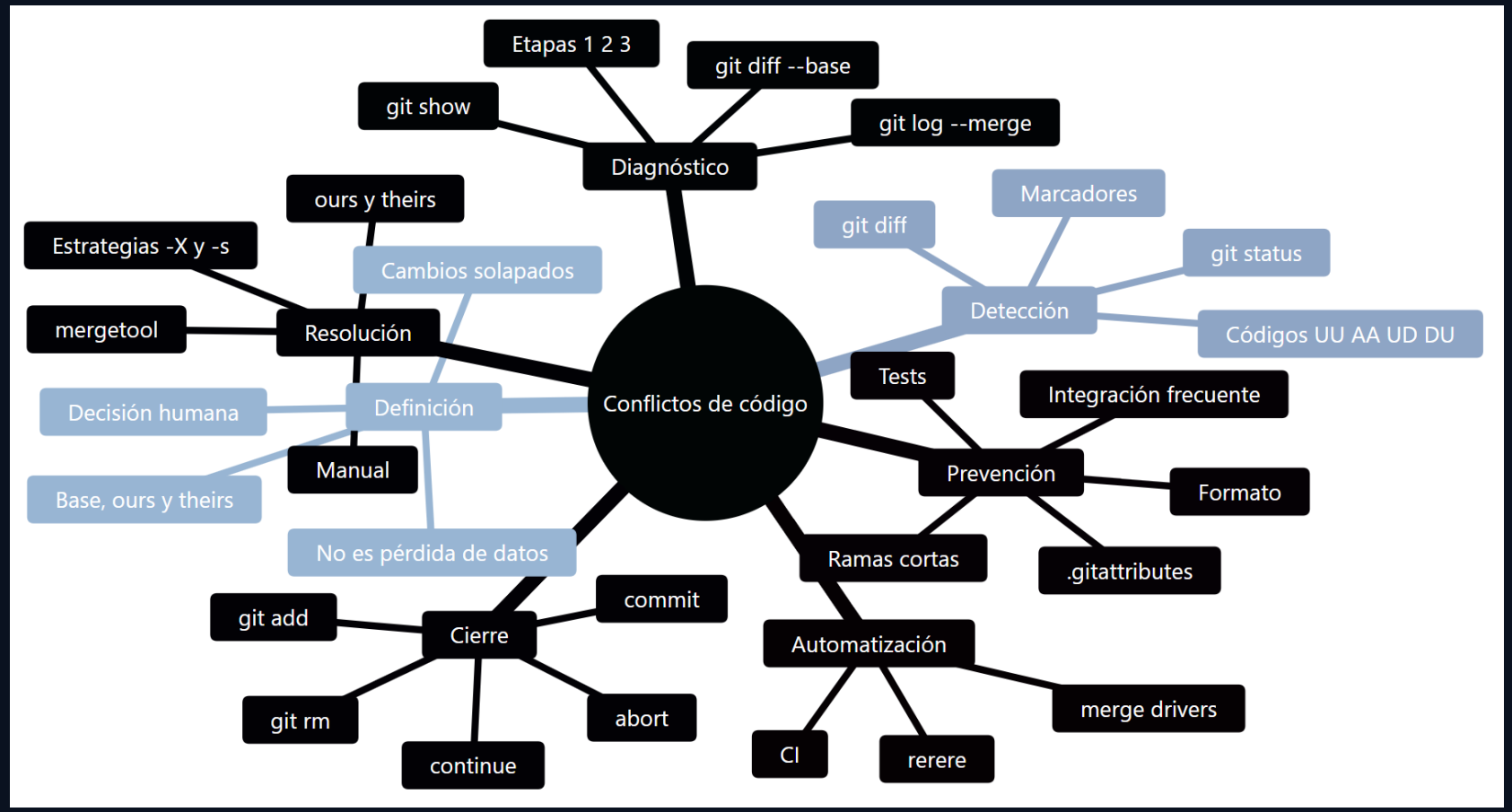
theirs: la otra rama

diff3 añade la base; zdiff3 la colapsa cuando coincide.

Con merge.conflictStyle=zdiff3 también ves la base, y entiendes qué cambió cada lado.

Del conflicto a la verificación

Seis nodos para no perder el hilo.



Detectar → entender → resolver → marcar → confirmar → verificar

Los estados que debes reconocer

git status --short es tu primer diagnóstico.

UU — **modificado por ambos**

Contenido en conflicto

```
git status --short  →  UU archivo
```

AA — **añadido por ambos**

Dos archivos nuevos

```
git status --short  →  AA archivo
```

UD / DU — **modificado / eliminado**

Una rama borró el archivo

```
git status --short  →  UD / DU archivo
```

DD — **eliminado por ambos**

Ambos lo borraron distinto

```
git status --short  →  DD archivo
```

UU es el más común. AA aparece cuando ambos crean el mismo archivo. UD/DU y DD requieren una decisión de producto.

Resolver en cinco pasos

Primero entender; después elegir; al final verificar.

01 **Detectar**

```
git status
UU app.txt
```

02 **Entender**

```
git diff
git show :1: :2: :3:
```

03 **Editar**

```
Abrir el archivo
combinar la intención
```

04 **Marcar**

```
git add app.txt
```

05 **Verificar**

```
git diff --check
git commit
```

Regla de oro No resuelvas un conflicto que no entiendes. Si el diff no tiene sentido, vuelve a la base y reconstruye las tres versiones.

Los 10 ejercicios

Todos ejecutados y verificados: 57 PASS · 0 FAIL.

01	Conflicto simple	5 PASS
02	Varios hunks	6 PASS
03	Modificar/eliminar	4 PASS
04	Añadir/añadir	5 PASS
05	Binario	4 PASS
06	Rebase	5 PASS
07	Cherry-pick	5 PASS
08	Stash pop	4 PASS
09	Inspección + mergetool	4 PASS
10	Estrategias + rerere	15 PASS

Cada práctica tiene script, log y captura. El repositorio permite repetirlas desde cero.

07 · LA TRAMPA

Ours y theirs no son lo que parecen

En un merge, ours eres tú. En un rebase, ours es la base.

MERGE

ours = tu rama actual (HEAD)
theirs = la rama que integras

:2: = ours · :3: = theirs

REBASE

ours = la rama base (upstream)
theirs = tu commit que se reaplica

:2: = base · :3: = tu commit

Trampa frecuente

Si usas --ours en un rebase, conservas la base y puedes perder el cambio que estabas reaplicando. No es “mi versión”: es la versión de la base.

La forma segura de comprobarlo

```
git show :2:archivo   # ours   ·   git show :3:archivo   # theirs
```

Conflicto simple en una línea

El caso base: mismo archivo, misma línea, dos intenciones.

Detección real

```
$ git merge feature
CONFLICT (content):
Merge conflict in app.txt

$ git status --short
UU app.txt

$ cat app.txt
<<<<<<< HEAD
linea 2 (main)
=======
linea 2 (feature)
>>>>>>> feature
```

1. Abrir app.txt y leer los dos lados.
2. Escribir la versión final (aquí, ambas).
3. git add app.txt y commit.
4. Verificar con git diff --check y git log.

01

simple

5 PASS **0 FAIL**

Salida real · la captura completa está en la documentación PDF

Lección: conservar ambas líneas es una decisión intencional, no un empate.

Casos que cambian la decisión

Varios hunks · modificar/eliminar · añadir/añadir.

02 · Varios hunks

Un archivo, dos regiones.

Resolver por bloque.
No aplicar --ours a todo.

6 PASS

03 · UD

feature borró notas.txt
main lo editó.

Conservar → git add
Eliminar → git rm

Se probaron ambas.

04 · AA

Ambas ramas crearon
CHANGELOG.md distinto.

No hay base común.
Unificar y git add.

5 PASS

La pregunta común: ¿qué debe existir y con qué contenido? Git muestra; la persona decide.

Binarios: no hay marcadores

Git no puede mostrar ni fusionar el contenido. Se elige una versión completa.

BIN

logo.bin

La resolución posible

```
$ git status --short
UU logo.bin

$ git diff
warning: Cannot merge binary files

$ git checkout --theirs -- logo.bin
$ git add logo.bin
$ git commit -m "merge: usa la version FEATURE"
```

Si necesitas conservar las dos versiones, renombra una antes de integrar. Git no va a inventar una combinación.

4 PASS

Rebase: la historia se reescribe

Tu commit se reaplica sobre otra base. Si choca, se detiene.

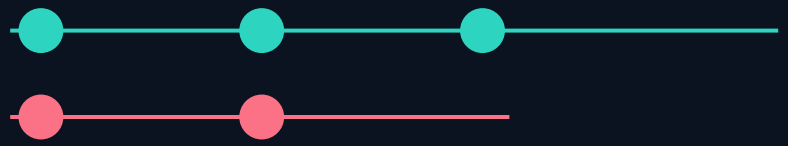
Secuencia

```
$ git switch feature
$ git rebase main
CONFLICT (content): version.txt
$ git show :2:version.txt   # base
$ git show :3:version.txt   # tu commit
$ git add version.txt
$ GIT_EDITOR=true git rebase --continue
```

OJO

En rebase, ours = base y theirs = tu commit. La etiqueta “ours” no significa “mi versión”.

ANTES



DESPUÉS



lineal, sin merge commit

5 PASS

Cherry-pick: un commit a la vez

Reaplicas un cambio concreto sobre otra base. El parche puede no encajar.

Secuencia

```
$ HASH=$(git rev-parse main)
$ git switch feature
$ git cherry-pick $HASH
CONFLICT (content): app.txt

$ git status
$ git add app.txt
$ GIT_EDITOR=true git cherry-pick --continue
```

Qué está pasando

Git copia el cambio del commit elegido y lo intenta aplicar sobre feature.

No crea un merge commit: reescribe un parche. Si la base cambió, hay conflicto.

Se resuelve, se marca y se continúa.

El conflicto es local a ese parche, no a toda la rama.

Cuidado: si el commit depende de otros cambios, el parche puede no compilar.

5 PASS

Stash: recuperar sin perder

El WIP choca con un commit nuevo. El stash sigue ahí hasta que lo resuelvas.



La secuencia segura

```
$ git stash pop
CONFLICT (content): app.txt
Updated upstream / Stashed changes

$ git add app.txt
$ git stash drop
$ git commit -m "feat: recupera el WIP"
```

Recuerda

Si pop conflictúa, el stash NO se elimina. Primero resuelves, luego haces drop y commit.

4 PASS

Inspección forense

Antes de editar, reconstruye las tres versiones.

:1:

BASE

Lo que había antes

:2:

OURS

Tu rama actual

:3:

THEIRS

La otra rama

Reconstruir

```
$ git show :1:config.txt
$ git show :2:config.txt
$ git show :3:config.txt
$ git log --merge --oneline
$ git diff --base
```

Mergetool

Una herramienta recibe BASE, LOCAL, REMOTE y MERGED. Puede ser visual (VS Code, meld, Beyond Compare) o un script propio. Git marca el archivo cuando la herramienta termina con éxito.

La base explica el "por qué" del conflicto. Sin ella, solo ves dos versiones y adivinas.

4 PASS

Estrategias automáticas

Atajos que reducen trabajo… y pueden ocultar decisiones.

`-X ours`

Fusiona, pero en los hunks conflictivos conserva ours. Los cambios no conflictivos de la otra rama sí entran.

Riesgo: pierdes hunks de la otra rama sin aviso.

`-s ours`

Crea un merge commit cuyo árbol es idéntico al de tu rama. Ignora todo el árbol de la otra.

Riesgo: descartas la otra rama por completo.

`merge=union`

Concatena las líneas de ambos lados. Útil en listas; peligroso en JSON o código.

Riesgo: archivo sintácticamente inválido.

`merge=ours`

Un driver que deja el archivo de ours. La otra versión se ignora sin conflicto.

Riesgo: cambios invisibles.

15 PASS Se probaron las cuatro variantes y se verificó qué se conservó y qué se descartó.

rerere y merge drivers

Recordar una resolución o delegar la fusión a una regla.

rerere

Reuse Recorded Resolution.

1. rerere.enabled true
2. conflicto y resolución
3. Git recuerda el conflicto
4. repites el merge
5. aplica la solución

Solo funciona con conflictos idénticos.

merge drivers

.gitattributes decide por ruta.

```
deps.lock merge=union
*.ours    merge=ours
```

union concatena; ours conserva el archivo de nuestra rama.

Potentes y silenciosos: documentar.

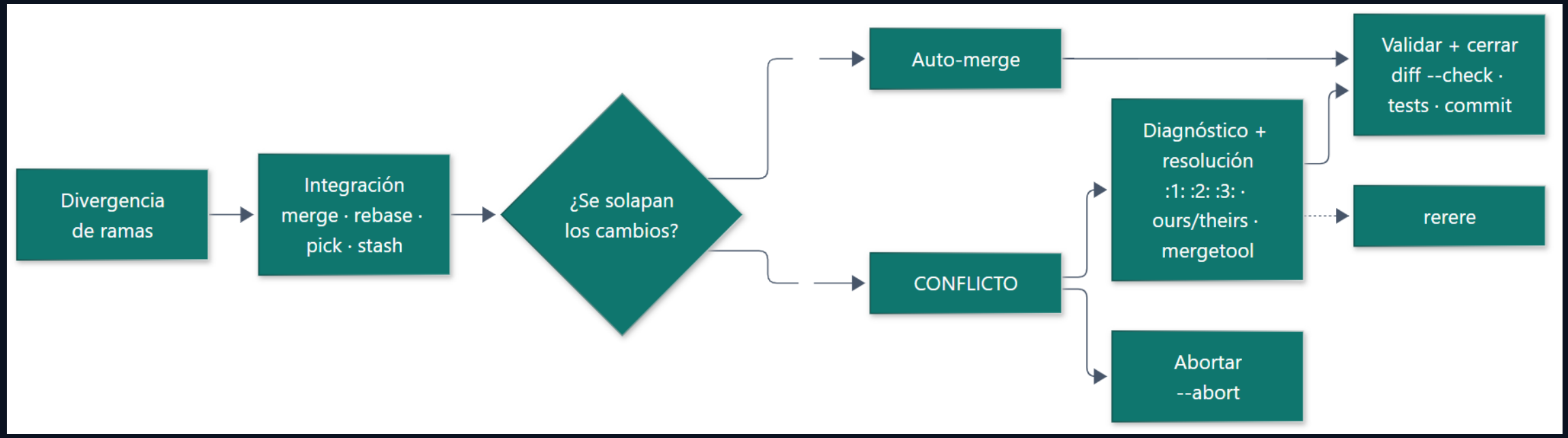
Configuración mínima

```
git config rerere.enabled true   ·   git config merge.ours.driver true
```

17 · MAPA CONCEPTUAL

Todo el sistema en una imagen

Del origen del conflicto a la decisión y la verificación.



Conflicto → diagnóstico → resolución → estrategia → validación

10/10 verificados

La práctica no termina cuando Git deja de quejarse: termina cuando las pruebas pasan.

10/10

ejercicios ejecutados

57

comprobaciones PASS

0

fallos

Repositorio público

github.com/75749748-blip/conflictos-git-ejercicios

Guía · PDF · PPT · 10 scripts · 10 logs · 23 capturas

Cada log termina en RESULTADO: N PASS, 0 FAIL. Cada captura sale de ese log.

Checklist para salir del conflicto

Entender, decidir, marcar y verificar.

- ¿Sé qué cambió cada lado y por qué?
- ¿Leí la base y las tres etapas?
- ¿Resolví por bloque, no por archivo?
- ¿Quité todos los marcadores?
- ¿Ejecuté git diff --check?
- ¿El resultado pasa las pruebas?

10/10	57	0
ejercicios	PASS	FAIL

Repositorio y evidencia:

github.com/75749748-blip/conflictos-git-ejercicios

Guía · PDF · PPT · 10 scripts · 10 logs · 23 capturas

Gracias.